An IMDb Dashboard

Lecture 2: Streamlit and SQL aggregates

Gregory S. DeLozier, PhD

Today

Layer	Responsibility
PostgreSQL	Store rows and execute SQL
database.py	Own queries and connections
app_basics.py	Controls, tables, charts

Exercise: Resume and Verify

```
bash start-postgres.sh
source .venv/bin/activate
export DATABASE_URL=postgresql://student@127.0.0.1:5433/imdb
python check_setup.py
```

Expected: imported tables and indexes are still present.

Exercise: Open the Starter

```
streamlit run app_basics.py --server.address 0.0.0.0
```

Open web port 8501. Leave database port 5433 local.

Starter: title search and yearly summaries.

Full app: Search, Notes, and graphical Reports.

One Parameterized Query

```
connection.execute(
    "SELECT primary_title FROM imdb_course.titles "
    "WHERE start_year = %s",
    (1994,),
)
```

SQL text and parameter values travel separately.

Connection and Display Lifecycle

Event	What happens
Query call	Open a connection; execute; close
Interface rerun	Reuse cached results when available
Refresh button	Clear the result cache

Cache rows; keep connection ownership simple.

A Join Produces Movie Rows

```
SELECT t.primary_title, t.start_year,
       r.average_rating, r.num_votes
FROM imdb_course.titles t
JOIN imdb_course.ratings r USING (title_id)
WHERE r.num_votes >= 1000;
```

The inner join leaves out unrated titles.

Grouping Produces Yearly Rows

```
SELECT t.start_year, count(*) AS movies,
       round(avg(r.average_rating), 2) AS mean_rating
FROM imdb_course.titles t
JOIN imdb_course.ratings r USING (title_id)
WHERE t.start_year IS NOT NULL AND r.num_votes >= 1000
GROUP BY t.start_year
ORDER BY t.start_year;
```

What the Dashboard Measures

Output	Meaning
movies	Qualifying titles in this sample
mean_rating	Equal-weight mean of title ratings
start_year	Reported release year

Source order and filtering affect the chart.

WHERE and HAVING

```
-- Keep rows before grouping.
WHERE r.num_votes >= 1000

-- Keep groups after aggregation.
HAVING count(*) >= 5
```

A group threshold belongs after GROUP BY.

Worked Example: Movie Length

Files: `database.py`, `app_basics.py`

Compare yearly counts and average ratings after excluding shorter movies.

Number input -> cached function -> SQL filter -> yearly report

The stored data stays unchanged.

1. Add the SQL Filter

In `database.py`, extend `yearly_summary`:

```
def yearly_summary(minimum_votes=1000, minimum_movie_length=0):
```

Add this condition before `GROUP BY`:

```
AND t.runtime_minutes >= %s
```

Replace the parameter tuple:

```
(minimum_votes, minimum_movie_length)
```

2. Add the Input

In `app_basics.py`, below the minimum-votes input:

```
minimum_movie_length = st.sidebar.number_input(
    'Minimum movie length (minutes)',
    min_value=0, value=0, step=10,
)
```

The selected number is a movie length in minutes.

3. Connect the Input to the Query

Replace the cached function in `app_basics.py`:

```
@st.cache_data(ttl=60)
def summary(votes, minimum_movie_length):
    return database.yearly_summary(votes, minimum_movie_length)
```

Inside `with second:`, replace the call:

```
rows = summary(votes, minimum_movie_length)
```

4. Compare the Results

Save both files; run or reload the starter:

```
streamlit run app_basics.py --server.address 0.0.0.0
```

Open **By year**. Keep votes fixed; try **0, 90, 120 minutes**.

- Counts can decrease; empty years disappear.
- Average ratings can rise or fall.
- Return to 90: the earlier results should return.

Your Turn: Maximum Movie Length

Files: `database.py`, `app_basics.py`

Add a **Maximum movie length (minutes)** input, initially **300**.

Pass it through the cache and apply an inclusive SQL upper bound.

Reject a maximum below the minimum before querying.

Check 90-120, 90-180, and 120-120 minutes.

The Working Boundary

SQL determines which rows and summaries mean something.

The data layer returns them.

Streamlit makes the result inspectable.

Next: rank and compare rows without collapsing them.